

Homework 3

Due: 11:59pm on April 8, 2025 (Tuesday)

Objectives:

- Implement your own Singly Linked List from scratch using recursion.
- Solidify your understanding of recursion.
- Have a better insight about Comparable and compareTo in Java.
- Take an interface and implement it on your own.

In Homework 1, you implemented the `MyArray` class to store words parsed from a text file.

In this homework, you will accomplish something similar, including adding new words, getting the current size of the list, removing words and displaying the current values, but you are to implement a different data structure class and use it.

```
public class SortedLinkedList implements MyListInterface
```

Your SortedLinkedList class must have the following properties.

- It must implement the `MyListInterface` that is provided to you.
- It does not allow any duplicates at any point. It needs to check duplicates as a new word is being added.
- It is a (singly) linked structure of nodes and each node's data type is `String`.
- It is **always** (or at any point) sorted / ordered in an ascending order. (For example, when you add "d", "c", "a" and "b" sequentially into the `SortedLinkedList` and then call the `display()` method of the list, you should see **[a, b, c, d]** as a result. The structures of the list should be ordered! Also, please refer to `HW3Driver.java` file to check the correct output of `display()` method.)

In addition to the properties above, methods in the `SortedLinkedList` **must** use recursion. In other words, there should be **NO loops** in your `SortedLinkedList` implementation, not even in its constructors. **Once again, not a single loop is allowed.**

Also, you are not allowed to have any other fields in `SortedLinkedList` class except the head reference as follows. Do not use Generics since the type of data in node is always expected to be `String`.

```
private Node head;
```

`MyListInterface` interface is where you can find necessary information about the methods to be implemented. **Please read the interface and its documentation very carefully.** As you override and implement the public interface methods, you may find it helpful to declare additional recursive private (helper) methods that take in more arguments.

Along with a constructor with no parameters, the `SortedLinkedList` class should have another constructor that takes unsorted `String` array as a parameter and build a new `SortedLinkedList`, as you can see from the example of calling this constructor in the `HW3Driver.java`. **Do not use any sorting algorithm on the input array!** The `add` method of the `SortedLinkedList` should take care of adding each string into the right position in the list so that the list always stays ordered.

As you add words, you still need to perform validations and gently ignore non-words. As in HW1, a word is a sequence of letters [a..zA..Z], does not include digits [0..9], and does not include other characters than the sequence of letters [a..zA..Z]. You do not need to perform this validation recursively; feel free to borrow the `isWord` method from `MyArray.java`.

You should see the same output in the expected output comment in `HW3Driver.java` file when you run the `HW3Driver` with the `prideprejudice.txt` file provided to you. When running the `HW3Driver` with the provided text file, your IDE may require you to put the text file into the project (root) folder, not in the `src` (sub-)folder.

The `display` method should print all the values in one line and each word should be comma separated. After each comma, put a space to make it easy to read (except for the last word). **Please refer to example output in the driver file.** If the list is empty, print square brackets (opening and closing with no spaces in them). Pay attention to how you implement the `display` method. It should be efficient. (Hint: Think about Lab 1.)

You are NOT allowed to use anything from the Collections Framework. Also, there should be NO imports in your code.

Deliverables:

- Your source code file. Make sure to include your Andrew ID and name in the header comments as an author tag. Submit your source code file (`SortedList.java`) using Autolab (<https://autolab.andrew.cmu.edu>). Do not zip it, and do not use package statements.
 - Using either single-line or multi-line comments, write how your recursion works for any method where recursion is used (e.g., outlining the base and recursive cases.)

Grading:

Autolab will grade your assignment as follows.

- Working code: 90 points
- Coding conventions: 10 points
 - We will deduct one point for each coding convention issue detected.

Autolab will show you the results and scores of its grading within approximately a few minutes of your submission. You may submit multiple times to correct any problems with your assignment. Autolab uses the last submission as your grade.

The Autolab score is not final. The TAs will look at your source code for correctness and design, and deduct points if applicable. The most important criterion is always correctness. It is also critical that your code is efficient and follows the specification properly. Additionally, it should be readable and well organized. This includes proper use of clear comments. Points will be deducted for poor design decisions and unreadable code, etc. Common sources of deductions are shown below. Note that this list is not exhaustive and is merely intended to draw your attention to parts easy to overlook with this assignment; you may lose points if the TAs find substantial issues with your code outside of this list.

10 pts

- Using classes from the Java Collections Framework
- Using fields other than the `Node` `head` reference
- Using iteration instead of recursion (for each iterative method)
- Re-creating instances of `SortedList` in recursive methods
- Using extra data structures in recursive methods

3-5 pts

- Unnecessarily using Java Generics for Node and SortedLinkedList
- Improperly declaring the Node class (e.g., no nesting, incorrect access modifiers)
- Declaring public helper methods
- Performing String concatenations within a loop
- Improperly using StringBuilder, including concatenations in the append method, re-creating StringBuilder for each recursive method call, etc.
- Missing corner cases for add and removeFirst methods (e.g., what happens if the list is empty)
- Performing double the amount of work to check for duplicates for the add method (hint: can this check be done while you are attempting to add?)
- Not validating words for the add and contains methods
- Not explaining the base- and recursive-cases for methods in comments
- Not removing dead code
- Ignoring compiler warnings

As mentioned in the syllabus, **late submissions will NOT be accepted**. If you have multiple versions of your code file, make sure you submit the correct version.